

Maximizing ARIA-CTR Performance through Kernel-Level Memory Optimization on the Apple M1 GPU

Siwoo Eum¹[0000–0002–9583–5427],
Minho Song²[0009–0007–6277–0069],
Minjoo Sim³[0000–0001–5242–214X],
Hwajeong Seo⁴[0000–0003–0069–9061]

Hansung University, Seoul (02876), South Korea

¹shuraatum@gmail.com, ²smino0906@gmail.com,
³minjoos9797@gmail.com, ⁴hwajeong84@gmail.com

Abstract. The emergence of CPU-GPU integrated System-on-Chip architectures like Apple’s M1 has made Unified Memory Architecture mainstream in computing. While UMA eliminated the CPU-GPU data copy overhead that was a major bottleneck in traditional dGPU systems, it shifted the optimization focus to kernel-internal memory access patterns. This paper analyzes how memory optimization strategies based on data access patterns affect cryptographic algorithm performance in SoC environments. We implemented the block cipher ARIA in Counter (CTR) mode on Apple M1 GPU using the Metal framework, designing six implementation configurations that systematically vary the storage locations of ARIA’s primary data structures—S-boxes and round keys—across device, threadgroup, and constant memory spaces. Experimental results confirmed that memory strategies optimized for data access patterns have a decisive impact on performance. A hybrid strategy placing S-boxes with irregular access patterns in threadgroup memory and round keys with uniform broadcast access patterns in constant memory achieved 451.93 MB/s throughput, representing 47.07% improvement over the baseline configuration. These findings demonstrate that precisely analyzing data access patterns and utilizing appropriate memory hierarchies remains key for maximizing GPU kernel performance even in UMA environments.

Keywords: ARIA · GPU Computing · Apple M1 · Unified Memory Architecture · Memory Optimization · Metal

1 Introduction

The exponential growth of digital data has highlighted the critical importance of data protection across modern society. As sensitive information is increasingly stored and transmitted digitally, cryptographic technologies ensuring data confidentiality, integrity, and availability have become fundamental cybersecurity

elements. The need to protect vast amounts of real-time data in cloud computing, big data analytics, and IoT environments has driven the development of high-performance cryptography beyond simple encryption implementations.

Graphics Processing Units (GPUs), specialized for large-scale parallel processing, have attracted attention as platforms for accelerating cryptographic operations [12, 6]. Recent computing architectures are experiencing a paradigm shift with CPU-GPU integrated System-on-a-Chip (SoC) like Apple’s M-series chips becoming mainstream in mobile, desktop, and laptop markets. These SoCs adopt fundamentally different memory models from traditional PC architectures, requiring reconsideration of existing optimization methodologies [7, 7].

Apple’s M1 chip adopts Unified Memory Architecture (UMA), where CPU and GPU share the same physical memory pool. This eliminates the explicit data copy overhead between CPU and GPU memory, which was a major bottleneck in discrete GPU systems [5]. However, UMA does not make all memory access costs uniform. Complex memory hierarchies still exist within the GPU, consisting of registers, threadgroup memory, constant memory cache, L1/L2 caches, and system RAM [12, 16]. This raises important questions: how dependent is GPU kernel performance on internal memory hierarchy when host-device transfer costs are eliminated? Are traditional GPU optimization techniques still valid in new architectures like M1?

The core paradox is that while UMA freed programmers from explicit memory copying burdens, kernel-internal memory access latency has emerged as the decisive performance factor. The optimization battlefield has shifted from PCIe buses to subtle GPU core-memory interactions. Some research showing performance degradation compared to dGPUs despite UMA demonstrates potential pitfalls when UMA is regarded as ‘one giant memory pool’ while ignoring internal hierarchical characteristics [8].

Based on these issues, this paper explores the impact of memory strategies on performance when implementing and optimizing the ARIA block cipher in Apple M1 GPU environment. Section 2 describes ARIA algorithm, CTR mode, and M1 GPU architecture. Section 3 explains six ARIA-CTR implementation strategies. Section 4 presents experimental methodology, results, and analysis. Section 5 summarizes conclusions and future directions.

2 Related Work

2.1 ARIA Block Cipher [10]

ARIA is a 128-bit block cipher algorithm jointly developed in 2003 by cryptographic experts from academia, research institutions, and industry under the leadership of the National Security Research Institute (NSRI) of South Korea. It was established as a Korean standard (KS X 1213) in 2004 and was subsequently published as RFC 5794 [9] by the IETF (Internet Engineering Task Force), securing international recognition [13].

ARIA is based on a Substitution-Permutation Network (SPN) structure similar to AES algorithm, which is currently an international standard. The SPN

structure is a method that makes the relationship between input and output nonlinearly complex by repeatedly applying substitution and permutation operations to data blocks. ARIA has a fixed block size of 128 bits and supports three key lengths: 128, 192, and 256 bits. Depending on the key length used, it performs 12, 14, or 16 rounds of encryption processes respectively, with a unique 128-bit round key used for each round.

Each round of ARIA consists of three core operation layers:

- **Round Key Addition:** This is the process of performing bitwise XOR ($\oplus$) operation between the 128-bit (16-byte) intermediate data state and the 128-bit round key assigned to that round. This is the most fundamental operation that makes the security of the cipher dependent on the key.
- **Substitution Layer:** This is a core operation that provides non-linearity to the cipher and satisfies the confusion property. It substitutes the 16-byte state on a byte-by-byte basis using S-boxes (Substitution-boxes), which are 8-bit input/output lookup tables. ARIA uses a total of four different S-boxes ($S_1, S_2, S_1^{-1}, S_2^{-1}$), applying them in different combinations for odd and even rounds to break symmetry and enhance cryptographic strength.
- **Diffusion Layer:** This is a linear operation layer that provides the diffusion property of the cipher. It ensures that a single bit change in the state affects multiple bits in the next round, making statistical analysis difficult. ARIA performs diffusion by multiplying the state vector with a fixed 16×16 binary matrix, and this matrix multiplication is actually implemented efficiently as a combination of simple XOR operations [15].

Memory Access Pattern Analysis The performance of GPU kernels is largely determined by how well the algorithm’s memory access patterns align with the characteristics of the GPU’s memory subsystem. The core data requiring memory access in ARIA—S-boxes and round keys—exhibit very contrasting access patterns.

- **S-box:** Access to S-box tables in the substitution layer is determined by the data being encrypted, i.e., the byte values of the state. This is data-dependent access. When hundreds of threads within a GPU threadgroup simultaneously process different data blocks, each thread’s state values are likely to be different. Therefore, simultaneous access to S-box tables exhibits an **irregular scatter/gather** pattern distributed across the entire memory address space. Moreover, this S-box lookup operation occurs very **frequently**, hundreds of times per block (e.g., for 128-bit keys: 12 rounds $\times$ 16 bytes = 192 times).
- **Round Key:** In round key addition operations, all threads processing a specific round (e.g., round 3) read and use the **identical** round 3 key value. While the access location moves sequentially to the next round key as rounds progress, within a single round, all threads’ memory accesses concentrate on a single address, showing a very **uniform** pattern. This corresponds to a typical broadcast access pattern.

The fundamentally different access patterns of S-boxes and round keys strongly suggest that storing them in the same type of memory may be inefficient. Irregular and frequent access is suited for low-latency memory close to the cores, while uniform broadcast access benefits from utilizing specialized hardware caching and delivery mechanisms. Therefore, optimal performance requires understanding the access patterns of operations that require memory access in algorithm design and placing them in specialized memory spaces that match those patterns.’

2.2 Counter Mode

Block Cipher Mode of Operation defines methods for encrypting arbitrary-length data using block ciphers that process fixed-size blocks. CTR (Counter) mode is one such mode, characterized by its ability to use a block cipher like a stream cipher.

The operating principle of CTR mode is as follows. For each plaintext block to be encrypted, an input block is created by combining a nonce (a random value that is not reused) with a sequentially incrementing counter value. This input block is encrypted with the encryption key to generate a keystream block. Finally, this generated keystream is XORed bitwise with the corresponding plaintext block to obtain the final ciphertext block [1]. Decryption follows exactly the same process as encryption: XORing the ciphertext with the same keystream restores the original plaintext.

$$C_i = P_i \oplus E_K(\text{Nonce} \parallel \text{Counter}_i) \quad (1)$$

$$P_i = C_i \oplus E_K(\text{Nonce} \parallel \text{Counter}_i) \quad (2)$$

The most important and powerful characteristic of CTR mode is **parallelizability**. The process of computing the i -th keystream block has no correlation whatsoever with the computation of any other j -th keystream block. That is, the encryption process (more precisely, keystream generation) for all blocks can be performed completely independently of each other. This characteristic stands in stark contrast to CBC (Cipher Block Chaining) mode, which has sequential dependencies (e.g., the previous ciphertext block is used as input for the next plaintext block). The perfect parallelism of CTR mode ideally matches the GPU’s SIMT (Single Instruction, Multiple Threads) execution model, where thousands of threads can be assigned different counter values to simultaneously generate keystreams, thereby maximizing encryption throughput.

2.3 Apple M1 GPU Architecture and Metal Memory Model

Unified Memory Architecture(UMA) Apple’s M1 chip is a SoC with CPU cores, GPU cores, Neural Engine, and other I/O controllers highly integrated on a single silicon die. One of the defining features of this chip is its Unified Memory Architecture (UMA). In traditional dGPU systems, the CPU and GPU each

had separate physical memory (system DRAM and GPU VRAM) and communicated through the PCIe bus. This required explicit data copying using APIs like ‘cudaMemcpy’ before and after GPU work, causing significant latency and bandwidth bottlenecks in GPU-accelerated data processing.

In contrast, M1’s UMA allows all processing units to directly share LPDDR4X system RAM [7]. Since the CPU and GPU access the same physical address space, the concept of data copying itself disappears. Data written to memory by the CPU can be immediately read by the GPU without any transfer process, and vice versa. This provides many benefits including dramatically reduced data transfer latency, simplified programming models, and improved energy efficiency [5].

Memory of Metal Shading Language Metal is a low-level graphics and computing API provided for GPU programming on Apple platforms. MSL, Metal’s shader programming language, provides several address space qualifiers to specify the location and characteristics of memory where data is stored within GPU kernel code. Using the correct address space according to data access patterns is key to optimizing performance on the M1 GPU.

- **device**: Points to the main memory space accessible by GPU kernels. In a UMA environment, this corresponds to system RAM. It has the largest capacity and serves as the foundation for data sharing between CPU and GPU, but has the longest access latency as it is physically furthest from GPU cores. Buffers declared without special optimization are located in this space by default.
- **threadgroup**: Very fast on-chip memory shared by all threads within a single threadgroup (equivalent to CUDA blocks or OpenCL workgroups) [11]. This operates like scratchpad memory that programmers must explicitly load, store, and manage. Access latency is tens to hundreds of times shorter than **device** memory, making it ideal for caching frequently reused data within a threadgroup to minimize **device** memory access.
- **constant**: A memory space specially designed for read-only data that does not change during kernel execution. Data in this space is accessed through a separate read-only data cache, the constant cache. The most important feature of this memory space is hardware-level broadcast optimization. When all threads within a warp (typically a bundle of 32 threads) simultaneously attempt to read the same address in **constant** memory, the hardware processes this request with a single memory read and efficiently delivers the result to all threads. This greatly reduces memory bandwidth usage and provides significant performance improvements for uniform read access patterns [2, 17].
- **thread**: The fastest and smallest private memory space belonging to each individual thread. The MSL compiler primarily attempts to allocate variables declared in this space to the GPU’s physical registers.

3 Comparison of SoC GPU and dGPU

3.1 SoC GPU: Unified Memory Architecture (UMA)

SoC GPUs feature various processor cores including CPU, GPU, and Neural Engine integrated within a single chip or package. Apple’s UMA is a prime example that maximizes the advantages of this structure. UMA is an architecture where physically unified memory is shared by all processors in the system. Unlike traditional approaches where CPU and GPU have separate memory spaces, this eliminates the need for separate copy processes for data transfer. Consequently, data access latency is dramatically reduced and overall system power efficiency is improved. This can be understood as a concept of data ‘sharing’ rather than ‘movement’.

3.2 dGPU: Discrete and Software-level Unified Memory

dGPUs are mounted on independent processor boards together with VRAM. They connect to the system’s main memory(System RAM) through PCIe(Peripheral Component Interconnect Express) buses to exchange data. While this structure allows GPUs to secure large amounts of dedicated memory for extreme parallel computation, explicit or implicit data copying through the PCIe bus is essential during data exchange between CPU and GPU. NVIDIA’s CUDA Unified Memory is a software-level abstraction designed to address the programming inconvenience of this discrete structure. Memory allocated through APIs like *cudaMallocManaged* appears as a single address space to both CPU and GPU, but internally the data moves to the physical memory of the processor that needs it. While this improves programmer convenience, latency from data movement still exists due to hardware constraints.

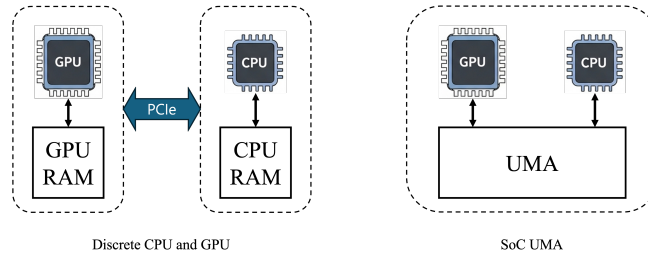


Fig. 1. Schematic diagram of the memory architecture of dGPU and SoC GPU. The left side shows a separated structure where the CPU and GPU each use independent memory (System RAM, VRAM), and the right side shows a unified memory structure where they share a single physical memory.

SoC GPUs and dGPUs are architectures that have evolved based on different design philosophies. SoC GPUs focus on overall system efficiency and

responsiveness. Through unified memory structures, they demonstrate excellent performance in complex workflows like video editing and graphic design where CPU and GPU must work organically together. Their advantages are particularly maximized in mobile and laptop environments where power consumption is limited. In contrast, dGPUs target maximum peak throughput. They provide overwhelming performance in tasks such as large-scale AI model training or scientific simulations that require housing vast amounts of data within the GPU and performing extreme parallel computations independently.

4 Implementation Strategy of ARIA-CTR on M1 GPU

The core hypothesis of this study is that "memory space selection optimized for data access patterns remains an important factor for performance optimization even in UMA environments." To experimentally verify this, we design and implement six specific implementation models that systematically vary the memory storage locations of the two key data structures in the ARIA encryption algorithm: S-box tables and round keys.

Previous research in dGPU environments has extensively analyzed the impact of memory hierarchy utilization on performance. In particular, it has been proven that simply moving frequently accessed lookup tables like S-box tables from global memory to shared memory can achieve significant performance improvements [14, 4, 3].

This study aims to empirically verify whether such memory optimization strategies remain equally effective in integrated GPUs in SoC environments. In UMA environments, since CPU and GPU share the same physical memory, they may exhibit different characteristics from traditional dGPU memory hierarchies. Therefore, the core of this research is to confirm whether memory access pattern optimization still has a decisive impact on performance in such environments.

All models are written using Metal Shading Language(MSL) and are based on ARIA-128-CTR.

Table 1. ARIA Kernels: Memory Usage Comparison

Type	Kernel Name	S-box	Round Keys
1	device_kernel	device	device
2	threadgroup_kernel	threadgroup	device
3	constant_kernel	constant	device
4	threadgroup_kernel_threadgroup_rk	threadgroup	threadgroup
5	threadgroup_kernel_constant_rk	threadgroup	constant
6	device_kernel_constant_rk	device	constant

4.1 Type 1: S-box in Device, Round Keys in Device

This implementation represents the most straightforward and intuitive approach. All data required for ARIA encryption is placed in the default memory space, ‘device’ memory. Specifically, both the S-box tables and the array of 13 128-bit round keys generated from the 128-bit key are passed to the kernel as ‘MTL-Buffer’ residing in ‘device’ memory. The GPU encryption kernel directly queries S-boxes in ‘device’ memory for every round and every byte substitution operation, and also directly reads round keys from ‘device’ memory for round key addition.

The goal is to measure performance when no additional optimizations are performed on the GPU’s sophisticated internal memory hierarchy. The performance measurements of this model serve as a clear baseline for comparing and evaluating the performance improvements of other optimization models. This enables quantitative isolation and analysis of the pure effects of each optimization technique applied in subsequent models.

4.2 Type 2: S-box in Threadgroup Memory, Round Keys in Device

This implementation applies the most classic and effective technique for optimizing lookup table-based algorithms in dGPU environments to the M1 environment [14]. It aims to improve performance by moving the S-box tables, which have the highest access frequency and most irregular access patterns during encryption operations, to high-speed on-chip ‘threadgroup’ memory. When the kernel begins execution, each threadgroup copies the original S-box table from ‘device’ memory to its ‘threadgroup’ memory space only once. After this initial copying operation is completed, all S-box lookup operations throughout the 12 rounds of encryption are performed through ‘threadgroup’ memory with extremely low access latency, rather than ‘device’ memory. Round keys remain in ‘device’ memory as in the baseline model.

The objective is to measure how effectively ‘threadgroup’ memory can resolve the performance bottleneck caused by the relatively high latency of ‘device’ memory due to frequent and irregular access patterns of S-boxes. Through performance comparison with Implementation 1, we can confirm whether ‘threadgroup’ memory utilization remains a core optimization strategy even in the M1 UMA environment.

4.3 Type 3: S-box in Constant Memory, Round Keys in Device

This implementation attempts to improve S-box lookup performance by utilizing constant memory specialized for read-only data access. Based on the characteristic that S-box tables never change during the encryption process, they are placed in the constant memory region. Round keys are maintained in device memory as in the baseline model (Implementation 1). The GPU kernel queries S-boxes located in constant memory for every round and every byte substitution operation, benefiting from caches optimized for constant memory. Round keys are read directly from device memory as before.

The goal of this implementation is to evaluate how efficiently S-box access patterns can be processed through constant memory’s caching mechanisms. While S-box lookups are read-only, access locations vary depending on the content of data blocks being encrypted, potentially causing divergent access patterns between threads. The purpose is to confirm how much performance improvement constant memory cache provides for such irregular read access compared to direct ‘device’ memory access (Type 1), and what performance differences exist compared to explicitly managed ‘threadgroup’ memory (Type 2).

4.4 Type 4: S-box in Threadgroup Memory, Round Keys in Constant Memory

This implementation uses a hybrid optimization strategy that assigns the most suitable memory type for each data’s access pattern characteristics. S-boxes with irregular and frequent access patterns are placed in threadgroup memory as in Implementation 2 to minimize access latency. Simultaneously, round keys, which are read-only and commonly used by all threads, are placed in constant memory. When the kernel executes, threadgroups copy S-boxes to threadgroup memory and access round keys through constant memory to perform round key addition operations.

The goal of this implementation is to verify how effective customized memory hierarchy utilization based on data access patterns can be. It aims to confirm whether synergy can be created by combining the advantages of threadgroup memory, which is strong against irregular access, and constant memory optimized for read-only uniform access. In particular, the purpose is to explore the optimal combination that most effectively utilizes the characteristics of each memory layer in GPU architecture by comparing and analyzing performance changes when round keys are placed in constant memory instead of device memory (Type 2) or threadgroup memory (Type 5).

4.5 Type 5: S-box in Threadgroup Memory, Round Keys in Threadgroup Memory

This implementation represents the most aggressive optimization approach that places both core data required for encryption operations—S-boxes and round keys—in threadgroup memory to maximize performance. When the kernel starts, each threadgroup copies both the original S-box tables and all 13 round keys from device memory to its threadgroup memory space. Subsequently, byte substitution and round key addition operations in all rounds are performed exclusively within ultra-fast threadgroup memory, with no device memory access occurring except during the initial copying phase.

The purpose of this implementation is to measure the maximum performance improvement achievable when all frequently accessed data is moved to the fastest programmer-managed memory. In particular, it aims to confirm whether moving

not only S-boxes with irregular access patterns but also round keys with relatively regular access patterns to threadgroup memory provides additional performance benefits. Through performance comparison with Type 2(using threadgroup memory only for S-boxe), it quantitatively evaluates whether the initial copying overhead and increased memory usage of moving round keys to threadgroup memory is worthwhile.

4.6 Type 6: S-box in device Memory, Round Keys in Constant Memory

This implementation employs a selective optimization strategy to isolate and measure the effect of optimizing round key access. Of the algorithm’s core data, the S-box tables remain in default device memory, identical to Type 1, while only the round keys, which are read-only and accessed uniformly, are moved to constant memory. When the kernel executes, S-box lookups for byte substitution operations are performed by directly accessing device memory, whereas round key addition operations are performed via the caching mechanism of constant memory.

The primary objective of this implementation is to quantitatively isolate and evaluate the impact of round key access pattern optimization on overall performance. By intentionally leaving S-box access unoptimized(using device memory), this model measures the performance improvement gained solely from optimizing round key access.

A performance comparison with Type 1(Device/Device) directly shows how much performance gain can be achieved by only changing round key access from device to constant memory. Furthermore, a comparison with Type 3(Constant/Device) provides a basis for analyzing whether the constant memory cache is more effective for the irregular access of S-boxes(Type 3) or the uniform access of round keys(Type 6).

5 Experimental Results and Analysis

5.1 Experimental environment and performance measurement method

This chapter quantitatively measures and comparatively analyzes the performance of the five GPU implementation models of the ARIA encryption algorithm designed in Chapter 3. The objective is to confirm the actual degree of performance improvement that memory optimization techniques applied to each implementation model bring in Apple M1’s Unified Memory Architecture(UMA) environment and to evaluate the effectiveness of each technique.

The detailed performance measurement environment is shown in Table 2.

The encryption processing performance of each implementation model was measured by distributing computations to the GPU through the Metal framework. The ultimate goal of this experiment is to perform a total of 2^{35} ARIA-CTR-128 encryption operations. To efficiently handle this massive workload, the GPU kernel was executed with the following configuration:

Table 2. Experimental Environment for Performance Measurement

Category	Specification
Model	MacBook Pro (MacBookPro17,1)
Chip	Apple M1
CPU	8-core (4 performance + 4 efficiency)
GPU	Apple M1 8-core integrated GPU
Memory	8 GB Unified Memory (UMA)
Operating System	macOS 15.5 Sequoia (Build 24F74, ARM64)
Development Language	Swift 6.1.2
GPU Framework	Metal (Version 32023.620)

- **Threads Per Threadgroup:** `MTLSize(width: 512, height: 1, depth: 1)`
1) One threadgroup consists of 512 threads.
- **Threadgroups Per Grid:** `MTLSize(width: 1024, height: 1, depth: 1)`
1) The entire grid consists of 1024 threadgroups.
- **Total Threads:** $512 \times 1024 = \mathbf{524,288 \text{ threads}}$

This means that a total of 524,288 threads execute simultaneously in a single kernel call. The entire workload of 2^{35} block encryptions is evenly distributed among these 524,288 threads. All threads encrypt different data blocks without duplication, effectively utilizing the parallelization characteristics of counter mode. Each thread was implemented to repeatedly perform encryption operations within a loop until completing its assigned workload.

The core performance metric was set as the **Total Execution Time** required for each implementation model to encrypt the entire workload (2^{35} blocks), using **seconds(s)** as the unit. Throughput(MB/s) was calculated from the measured units for performance comparison and analysis.

5.2 Performance Analysis

Table 3. Performance comparison of different memory configurations for ARIA-CTR

Type	S-box	Round Keys	Time (s)	MB/s
Type 1	Device	Device	106.64	307.32
Type 2	Threadgroup	Device	96.75	338.70
Type 3	Constant	Device	115.08	284.73
Type 4	Threadgroup	Threadgroup	73.58	445.12
Type 5	Threadgroup	Constant	72.52	451.93
Type 6	Device	Constant	80.52	406.90

The experimental results demonstrate significant performance variations based on memory placement strategies:

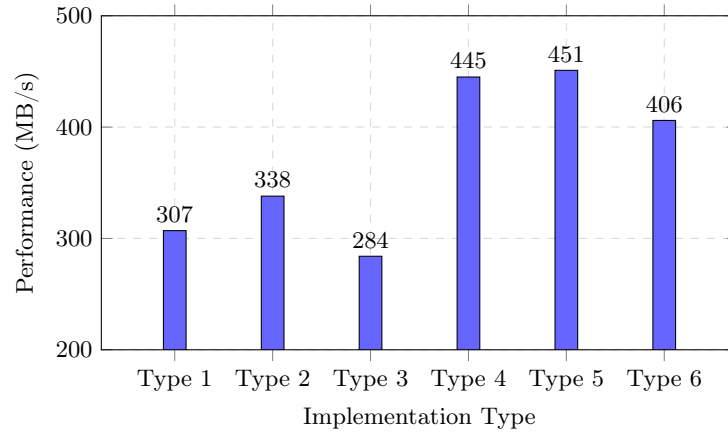


Fig. 2. Performance comparison of different memory configurations for ARIA-CTR

Best Performance Configuration(Type 5): The optimal configuration achieved 451.93 MB/s throughput with S-boxes in shared memory and round keys in constant memory. This configuration completed the workload in 72.52 seconds, representing a 47.07% improvement over the baseline Type 1 configuration.

Impact of S-box Memory Placement: Comparing Types 1, 2, and 3 (all with round keys in device memory), we observe:

- Device memory S-boxes (Type 1): 307.32 MB/s
- Threadgroup memory S-boxes (Type 2): 338.70 MB/s
- Constant memory S-boxes (Type 3): 284.73 MB/s

Interestingly, threadgroup memory configuration outperformed both device and constant memory when round keys remained in device memory, showing a 10.2% improvement over the baseline device memory configuration.

Impact of Round Key Memory Placement:

When S-boxes are in threadgroup memory (Types 2, 4, 5):

- Round keys in constant memory (Type 5): 451.93 MB/s
- Round keys in threadgroup memory (Type 4): 445.12 MB/s
- Round keys in device memory (Type 2): 338.70 MB/s

When S-boxes are in device memory (Types 1, 6):

- Round keys in constant memory (Type 6): 406.90 MB/s
- Round keys in device memory (Type 1): 307.32 MB/s

This clearly demonstrates that constant memory is the optimal location for round keys, providing consistent performance improvements of 33.4% (threadgroup S-box) and 32.4% (device S-box) over device memory placement.

The experimental results clearly demonstrate that memory utilization strategies optimized for data access patterns have a decisive impact on performance in modern SoC environments, just as in discrete GPU (dGPU) environments. In particular, for operations with irregular memory access patterns such as S-box substitution, where each thread references different locations in the table, we confirmed that using threadgroup memory with short access latency and fast intra-threadgroup data sharing is most effective. Conversely, round keys exhibit uniform access patterns where all threads within a warp simultaneously reference the same values. Storing pre-computed round key values in constant memory enables aggressive utilization of the constant cache’s broadcast functionality, serving as a key factor in maximizing memory access efficiency and increasing overall throughput.

6 Conclusion

This study aimed to experimentally verify the impact of memory utilization strategies optimized for data access patterns on high-performance cryptographic kernel implementation in Unified Memory Architecture (UMA) environments such as Apple’s M1. By implementing six configurations with different storage locations for S-boxes and round keys using the ARIA-128-CTR algorithm and measuring their performance, we clearly confirmed our core hypothesis that selecting memory spaces according to data access characteristics has a decisive impact on performance.

In particular, the hybrid strategy (Type 5) that places S-boxes with irregular and frequent references in **threadgroup** memory for high-speed data sharing within threadgroups, and round keys accessed simultaneously by all threads in **constant** memory to maximize hardware broadcast functionality benefits, achieved the highest throughput of 451.93 MB/s. This represents a 47.07% improvement over the baseline model (Type 1) with no memory optimization, suggesting that traditional GPU memory optimization principles remain valid and even more important amid the structural changes brought by UMA.

The most significant contribution of this study lies in empirically demonstrating that while UMA eliminated the macroscopic bottleneck of CPU-GPU data transfer, kernel-internal memory access latency has emerged as the key factor determining overall performance. This research shows that developers should not simply regard UMA as "one giant single memory pool," but rather must develop a deeper understanding of the subtle interactions between GPU cores and memory hierarchies, precisely analyze data access patterns, and make essential efforts to place data in optimal memory spaces.

Future research could verify generalizability by applying the same optimization methodology to other symmetric or public key cryptographic algorithms besides ARIA, or track performance changes in subsequent chip architectures such as M2 and M3 released after M1. Additionally, analyzing the interactions between memory strategies and other optimization variables such as threadgroup size or data placement would constitute meaningful follow-up research.

References

1. An, S., Kim, Y., Kwon, H., Seo, H., Seo, S.C.: Parallel implementations of arx-based block ciphers on graphic processing units. *Mathematics* **8**(11), 1894 (2020)
2. Apple Inc.: Advanced metal shader optimization. WWDC 2016, Session 606 (2016), <https://developer.apple.com/videos/play/wwdc2016/606/>, accessed: November 24, 2025
3. Eum, S., Kim, H., Kwon, H., Sim, M., Song, G., Seo, H.: Parallel implementations of aria on arm processors and graphics processing unit. *Applied Sciences* **12**(23), 12246 (2022)
4. Eum, S., Song, M., Kim, S., Seo, H.: Efficient gpu parallel implementation and optimization of aria for counter and exhaustive key-search modes. *Electronics* **14**(10), 2021 (2025)
5. Gebraad, L., Fichtner, A.: Seamless gpu acceleration for c++-based physics with the metal shading language on apple's m series unified chips. *Seismological Society of America* **94**(3), 1670–1675 (2023)
6. Go, Y., Jamshed, M.A., Moon, Y., Hwang, C., Park, K.: {APUNet}: Revitalizing {GPU} as packet processing accelerator. In: 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). pp. 83–96 (2017)
7. Hübner, P., Hu, A., Peng, I., Markidis, S.: Apple vs. oranges: Evaluating the apple silicon m-series socs for hpc performance and efficiency. *arXiv preprint arXiv:2502.05317* (2025)
8. Jarzabek, Ł., Czarnul, P.: Performance evaluation of unified memory and dynamic parallelism for selected parallel cuda applications. *The Journal of Supercomputing* **73**, 5378–5401 (2017)
9. Kim, J., Lee, J., Kim, C., Lee, J., Kwon, D.: A Description of the ARIA Encryption Algorithm. RFC 5794 (Mar 2010). <https://doi.org/10.17487/RFC5794>, <https://www.rfc-editor.org/info/rfc5794>
10. Kwon, D., Kim, J., Park, S., Sung, S.H., Sohn, Y., Song, J.H., Yeom, Y., Yoon, E.J., Lee, S., Lee, J., et al.: New block cipher: Aria. In: International conference on information security and cryptology. pp. 432–445. Springer (2003)
11. Lee, W.K., Goi, B.M., Phan, R.C.W., Poh, G.S.: High speed implementation of symmetric block cipher on gpu. In: 2014 International Symposium on Intelligent Signal Processing and Communication Systems (ISPACS). pp. 102–107. IEEE (2014)
12. Li, Q., Zhong, C., Zhao, K., Mei, X., Chu, X.: Implementation and analysis of aes encryption on gpu. In: 2012 IEEE 14th international conference on high performance computing and communication & 2012 IEEE 9th international conference on embedded software and systems. pp. 843–848. IEEE (2012)
13. Seo, H., Kwon, H., Kim, H., Park, J.: Ace: Aria-ctr encryption for low-end embedded processors. *Sensors* **20**(13), 3788 (2020)
14. Tezcan, C.: Optimization of advanced encryption standard on graphics processing units. *IEEE Access* **9**, 67315–67326 (2021)
15. Wu, W., Zhang, W., Feng, D.: Impossible differential cryptanalysis of aria and camellia. *Cryptology ePrint Archive* (2006)
16. Yi, X.: A study of performance programming of cpu, gpu accelerated computers and simd architecture. *arXiv preprint arXiv:2409.10661* (2024)
17. Yuan, S., Awad, A., Yudha, A.W.B., Solihin, Y., Zhou, H.: Adaptive security support for heterogeneous memory on gpus. In: 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA). pp. 213–228. IEEE (2022)